

★ SPECIAL • LECTURE

Recommendation Systems

Content-based, collaborative filtering — and matrix factorization

The third running app of this course, given its own session. How Netflix, Amazon, and your bank decide what to suggest next — built from ideas you already know (PCA, embeddings).

Where We Are Going Today

Two families of recommender, plus the secret sauce

1

What is a recommender?

The smart shop assistant who remembers everyone's taste. Why finance cares.

2

Two ways to recommend

By the ITEM (content-based) and by the PEOPLE (collaborative filtering).

3

Matrix factorization + doing it right

Hidden taste factors (like PCA & embeddings), the cold-start problem, and honest evaluation.

Recommendation reuses old friends: cosine similarity (W12), PCA (W9), and honest testing (W10).

1

What Is a Recommender?

The assistant who knows everyone's taste

You Meet Recommenders Every Day

'Customers who bought this also bought...'

Netflix suggests your next show; Amazon suggests your next purchase; Spotify builds your playlist.

Each is a recommender: given who you are and what you've liked, it predicts what you'll like next.

What it really does

- Predict a rating you HAVEN'T given yet.
- Then suggest the items with the highest predicted rating.
- It fills in the blanks of 'what would you think of this?'

Why finance cares

- Which fund, card, or insurance to offer a client?
- 'Next best action' and cross-selling.
- Robo-advisors: match portfolios to people.

Two Big Families of Recommender

Match by the item, or match by the people

Content-based

- Looks at the ITEM's features.
- 'You liked this, here's a similar one.'
- Needs good item descriptions.
- Works even with few users.

Collaborative filtering

- Looks at the PATTERN of who liked what.
- 'People like you also liked...'
- Needs many users and ratings.
- Finds surprising picks the features would miss.

Most real systems blend BOTH (a 'hybrid'), so each covers the other's weakness.

2

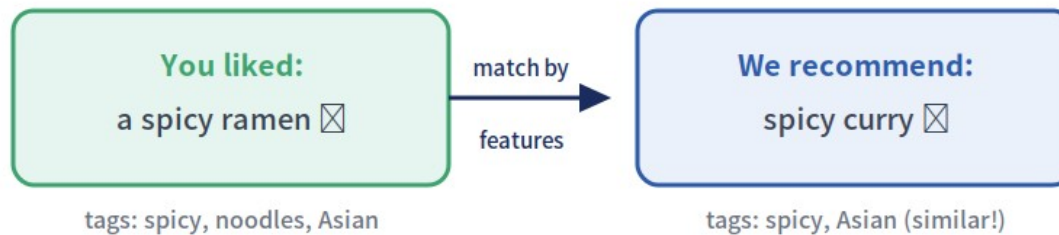
Recommend by the Item

Content-based filtering

Content-Based — Match Items by Features

Describe each item by its features, then find the closest ones

Content-Based — "You Liked This, Here's a Similar One"



How it works

Describe each item by its FEATURES (genre, spice level, price...).

Find items whose features are CLOSE to what you already liked (cosine similarity).

No other users needed — just the item descriptions and your own history.

No other users needed — just the item descriptions and your own history.

Measuring 'Similar Items'

The same cosine similarity from Week 12

$$\text{similarity} = \cos(\theta) = \frac{a \cdot b}{\|a\| \|b\|}$$

Reading it

- a and b are two items' feature vectors.
- Cosine measures the ANGLE between them.
- Near 1 = very similar items; near 0 = unrelated.

Strengths & limits

- + No cold-start for new items (use their features).
- + Easy to explain: 'similar because both spicy'.
- - Stuck in a bubble: only ever suggests look-alikes.
- - Needs good, clean feature data.

3

Recommend by the People

Collaborative filtering & matrix factorization

Collaborative Filtering — Lean on the Crowd

Find people with your taste; suggest what they loved that you haven't tried

Collaborative Filtering — "People Like You Also Liked..."

Who rated what (★ = liked, ? = unknown):

	Fund A	Fund B	Fund C	Fund D
You	★	★	?	.
Mia	★	★	★	.
Leo	★	.	★	.
Ann	.	.	.	★

You and Mia have nearly identical taste

The key idea

Ignore WHAT the items are. Just look at the pattern of who liked what.

You and Mia agree on Funds A and B.
Mia also loves Fund C — and you haven't tried it.

So we recommend Fund C to you.

"Wisdom of similar people" — the crowd guides you.

It ignores what items ARE — only the pattern of who liked what matters.

Two Flavors of Collaborative Filtering

Find similar users, or find similar items

User-based

- Find users whose ratings look like yours.
- Recommend what those neighbors liked.
- 'People like you also bought...'
- Intuitive, but slow with millions of users.

Item-based

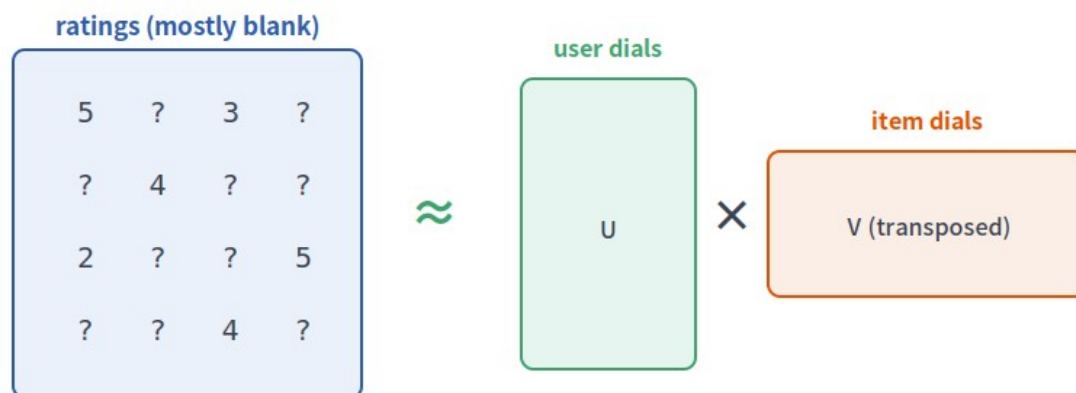
- Find items that get rated together.
- If you liked X, suggest items often liked with X.
- 'Bought together' on shopping sites.
- Usually faster and more stable in practice.

Both rely on similarity (cosine again). The next idea, matrix factorization, scales them up.

Matrix Factorization — Hidden Taste Factors

Explain millions of ratings with a few hidden 'dials'

Matrix Factorization — Discover Hidden "Taste Dials"



A few hidden factors explain it all

Maybe taste comes down to a few dials:
"likes risk?", "likes tech?", "likes income?"

Each user has dial settings; so does each item.
Multiply them to PREDICT the blank ratings.

It's the same idea as PCA (W9) and embeddings (W12): hidden factors, learned.

This powered the famous Netflix Prize — and it's PCA/embeddings wearing a new hat.

Matrix Factorization, in Symbols

Split the ratings into user and item factors

$$R \approx U V^{\top} \quad (\text{ratings} \approx \text{user-factors} \times \text{item-factors})$$

To predict one rating, take the dot product of that user's dials and that item's dials:

$$\hat{r}_{u,i} = \mathbf{u}_u \cdot \mathbf{v}_i$$

Reading it

- R is the big, mostly-blank ratings table.
- U = each user's hidden dials; V = each item's dials.
- Multiply to fill in the blanks — your predicted ratings.

The link to earlier weeks

- Hidden factors = exactly the PCA idea (Week 9).
- User/item vectors = embeddings (Week 12).
- You already understand the machinery.

The Cold-Start Problem

A brand-new user or item has no history to learn from

The Cold-Start Problem — a Brand-New Customer

The problem



A new customer has rated NOTHING yet.

Collaborative filtering has no pattern to use.

A brand-new item has no ratings either.

The assistant has nothing to go on.

Common fixes

Fall back to CONTENT-based (use features).

Ask a few onboarding questions.

Recommend popular items to start.

Use what you know: age, goals, region.

Then switch to collaborative as history grows.

The classic fix: start with content-based, then switch to collaborative as ratings pile up.

4

Doing It Right

Honest evaluation and finance uses

Evaluating a Recommender Honestly

The Week 10 lessons apply here too

It's just as easy to fool yourself here as with a trading backtest.

If you test on ratings the model already saw, it looks brilliant — and fails on real new users.

How to measure

- Hide some real ratings; predict them; check the hits.
- Precision@k: of the top k suggestions, how many landed?
- Always test on data the model never saw (no leakage).

Beyond accuracy

- Diversity: don't show ten near-identical items.
- Novelty: surface things they wouldn't find alone.
- Fairness (W14): don't trap groups in narrow bubbles.

Recommenders in Finance

Where 'you may also like' meets money

Product & cross-sell

- Suggest the next card, loan, or insurance.
- 'Next best action' for each client.
- Personalize the app and offers.

Robo-advisor

- Match portfolios to a client's goals & risk.
- Recommend funds similar to ones they hold.
- Blend content (fund features) + collaborative.

Research & news

- Surface filings and news like ones you read.
- Recommend stocks similar to your watchlist.
- Connects to embeddings & RAG (Week 12).

Same engine as Netflix — but the stakes are someone's savings, so honesty and fairness matter more.

Getting Started

A tiny ratings table, built right in the notebook

A built-in ratings matrix (no download)

- Rows = users, columns = items (funds).
- Numbers = ratings; zeros = not yet rated.
- Small enough to read by eye, real enough to learn.
- No internet or Kaggle needed.

What we'll do

- Lab 1: item-similarity recommender (cosine).
- Lab 2: matrix factorization to fill the blanks (SVD).
- All in numpy + scikit-learn, a few lines each.
- See both families of recommender in action.

colab_setup.py — tiny ratings matrix (built inline)

```
import numpy as np
# rows = 4 users, cols = 5 funds; 0 means 'not rated yet'
R = np.array([[5,4,0,0,1],
              [4,0,0,2,1],
              [1,1,0,5,4],
              [0,0,5,4,4]], float)
```

Hands-On Lab 1 — Item-Similarity Recommender

Find funds that are rated alike

Your task (step by step)

- 1. Treat each fund (column) as a vector of ratings.
- 2. Compute cosine similarity between funds.
- 3. Pick a fund a user liked.
- 4. Recommend the most similar fund.
- Question: do the suggestions make sense?

`colab_lab1.py` • `tiny ratings matrix (built inline)`

```
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

# items are columns -> compare funds by their rating columns
item_sim = cosine_similarity(R.T) # fund-by-fund similarity
np.fill_diagonal(item_sim, 0)     # ignore self-match

liked = 0                          # user liked fund 0
best = item_sim[liked].argmax()
print('you liked fund', liked)
print('most similar fund to recommend:', best)
```

Hands-On Lab 2 — Fill the Blanks (SVD)

Matrix factorization predicts unseen ratings

Your task (step by step)

- 1. Factor the ratings into hidden dials (SVD).
- 2. Keep just a few factors.
- 3. Multiply back to predict ALL ratings.
- 4. Read off a prediction for a blank cell.
- Question: what does it suggest a user hasn't tried?

`colab_lab2.py` • `tiny ratings matrix (built inline)`

```
import numpy as np
from sklearn.decomposition import TruncatedSVD

svd = TruncatedSVD(n_components=2, random_state=0)
U = svd.fit_transform(R)          # user dials
V = svd.components_               # item dials

R_hat = U @ V                     # predicted full ratings
user = 0
unseen = np.where(R[user] == 0)[0] # funds not yet rated
pick = unseen[R_hat[user, unseen].argmax()]
print('recommend fund', pick, 'to user', user)
```

Comprehension Check

Try first, then reveal the answer below

Q1 What does collaborative filtering rely on?

A The item's features

B The pattern of who liked what ✓

C The item's price only

D A single user's data

Answer: It uses the pattern of ratings across many users — 'people like you also liked...' — not item features.

Q2 Matrix factorization is most like which earlier idea?

A Purged cross-validation

B PCA / embeddings (hidden factors) ✓

C The Sharpe ratio

D Isolation Forest

Answer: It discovers a few hidden factors to explain the ratings — the same idea as PCA (W9) and embeddings (W12).

Three Ideas, One Picture Each

1

Content-based = match by features

Describe items; suggest ones similar to what you liked (cosine). Great for new items, but stays in a bubble.

2

Collaborative = match by people

Use the pattern of who liked what. 'People like you also liked...' Finds surprises, but needs many users.

3

Matrix factorization = hidden dials

Explain ratings with a few latent factors (PCA/embeddings). Powerful, scalable — watch the cold-start problem.

Recommendation = cosine similarity + hidden factors + honest testing — all tools you already had.

Special Session — Three Things to Remember

- 1 Two families, often blended**
Content-based (item features) and collaborative (user patterns). Hybrids combine their strengths.
- 2 Matrix factorization is the workhorse**
Find hidden taste factors and multiply to predict blanks. It's PCA & embeddings applied to ratings.
- 3 Test honestly, recommend responsibly**
Hide ratings to evaluate (precision@k, no leakage); value diversity and fairness, not just accuracy.

That completes all three running apps: fraud detection, the finance chatbot, and recommendation.

Try both labs, then swap in a bigger ratings matrix and see how the suggestions change.